Streamlit

Streamlit

- **Streamlit** is an open-source Python library for building interactive web apps using only Python.
- It's ideal for creating dashboards, data-driven web apps, reporting tools and interactive user interfaces without needing HTML, CSS or JavaScript.

Streamlit Installation

- Make sure Python and pip are already installed on your system. To install Streamlit, run the following command in the command prompt or terminal:

pip install streamlit

How to run Streamlit file?

- To run a Streamlit app, open the command prompt or Anaconda prompt and type:

streamlit run filename.py

- For example, if file name is sample.py, run:

streamlit run sample.py

- After running the command, Streamlit starts a local development server and displays a URL (http://localhost:8501). Open this URL in your browser to view and interact with the app.

Understanding Streamlit basic functions

- Streamlit offers simple built-in functions to create interactive web apps using Python. These functions help display text, add widgets, visualize data and handle user input.

1. Title

- This basic Streamlit example displays a title on the web page, confirming that Streamlit is set up and running correctly.

- import streamlit as st
- st.title("Hello India !!!")

2. Header and Subheader

- This example demonstrates how to add a header and subheader in a Streamlit app.

st.header("This is a header")

st.subheader("This is a subheader")

- **3. Text**
- It shows how to display plain text in a Streamlit app using st.text() function.

```
st.text("Hello India!!!")
```

- **4. Markdown**
- This example uses st.markdown() to show formatted text. The ### creates a level 3 header, used for medium-sized headings in the app.

st.markdown("### This is a markdown")

5. Success, Info, Warning, Error and Exception

- This example shows how to display different types of messages and alerts in a Streamlit app using built-in functions like st.success(), st.info(), st.warning(), st.error() and st.exception(). These functions help communicate status, information, warnings, errors and exceptions to users clearly.

```
st.success("Success")

st.info("Information")

st.warning("Warning")

st.error("Error")

exp = ZeroDivisionError("Trying to divide by Zero")
st.exception(exp)
```

6. Write

- This example uses st.write() that can display text, numbers, data structures and even charts. Here, it's used to show plain text and the output of Python’s built-in range() function.

```
st.write("Text with write")
# Writing python inbuilt function range()
st.write(range(10))
```

7. Display Images

- This example demonstrates how to display an image using Pillow library. The image is opened with Image.open() and displayed with st.image(), where the width parameter controls its size.

```
from PIL import Image  # Import Image from Pillow
img = Image.open("streamlit.png") # Open the image file
st.image(img, width=200) # Display the image with a specified width
```

- **8. Checkbox**
- This example uses a checkbox in Streamlit to toggle content visibility. When the checkbox labeled "Show/Hide" is checked, it displays a text message on the screen.

```
# Display a checkbox with the label 'Show/Hide'
if st.checkbox("Show/Hide"):
# Show this text only when the checkbox is checked
st.text("Showing the widget")
```

- **9. Radio Button**
- This example demonstrates how to use radio buttons to let users select one option from a list. Based on the selected gender, the app displays the result using **st.success()**

```
# Create a radio button to select gender
status = st.radio("Select Gender:", ['Male', 'Female'])
# Display the selected option using success message
if status == 'Male':
   st.success("Male")
else:
    st.success("Female")
```

10. Selection Box

- This example uses a select box in Streamlit to let users choose one option from a dropdown list. The selected item is then displayed on the screen.

```
# Create a dropdown menu for selecting a hobby

hobby = st.selectbox("Select a Hobby:", ['Dancing', 'Reading', 'Sports'])

# Display the selected hobby

st.write("Your hobby is:", hobby)
```

11. Multi-Selectbox

- This example demonstrates how to use a multiselect box in Streamlit, allowing users to choose multiple options from a list. The app then displays the number of selected items.

```
# Create a multiselect box for choosing hobbies

hobbies = st.multiselect("Select Your Hobbies:", ['Dancing', 'Reading', 'Sports'])

# Display the number of selected hobbies

st.write("You selected", len(hobbies), "hobbies")
```

12. Button

- This example shows how to use buttons in Streamlit. Buttons can trigger specific actions when clicked, such as displaying a message or running a function.

```
# A simple button that does nothing
st.button("Click Me")
# A button that displays text when clicked
if st.button("About"):
    st.text("Welcome to INDIA!")
```

13. Text Input

- Text input fields allow users to enter custom data. This example collects a user's name, formats it with proper capitalization and displays it when Submit button is clicked, simulating a basic form interaction.

```
# Create a text input box with a default placeholder
name = st.text_input("Enter your name", "Type here...")
# Display the name after clicking the Submit button
if st.button("Submit"):
 result = name.title()  # Capitalize the first letter of each word
st.success(result)
```

14. Slider

- Sliders provide a way to select numeric values within a range.
- This example lets users choose a level between 1 and 5 and displays the selected value instantly. It is useful for settings like ratings, difficulty levels or thresholds.
- # Create a slider to select a level between 1 and 5
- level = st.slider("Choose a level", min_value=1, max_value=5)
- # Display the selected level
- st.write(f"Selected level: {level}")

```
import streamlit as st
# Title of the app
st.title("BMI Calculator")
# Input: Weight in kilograms
weight = st.number_input("Enter your weight (kg):", min_value=0.0, format="%.2f")
# Input: Height format selection
height_unit = st.radio("Select your height unit:", ['Centimeters', 'Meters', 'Feet'])
# Input: Height value based on selected unit
height = st.number_input(f"Enter your height ({height_unit.lower()}):", min_value=0.0, format="%.2f")
# Calculate BMI when button is pressed
if st.button("Calculate BMI"):
    try:
        # Convert height to meters based on selected unit
        if height_unit == 'Centimeters':
            height_m = height / 100
        elif height_unit == 'Feet':
            height_m = height / 3.28
        else:
            height_m = height
```

```
# Prevent division by zero
        if height_m <= 0:
            st.error("Height must be greater than zero.")
        else:
            bmi = weight / (height_m ** 2)
            st.success(f"Your BMI is {bmi:.2f}")
# BMI interpretation
            if bmi < 16:
                st.error("You are Extremely Underweight")
            elif 16 <= bmi < 18.5:
                st.warning("You are Underweight")
            elif 18.5 <= bmi < 25:
                st.success("You are Healthy")
            elif 25 <= bmi < 30:
                st.warning("You are Overweight")
            else:
                st.error("You are Extremely Overweight")
    except:
        st.error("Please enter valid numeric values.")
```